R E V I E W – I R E P O R T

Problem Identification and Design

GraphRAG-Powered Cyber Threat Intelligence Platform: A Multi-Model NoSQL Approach

Short title: Multi-Model NoSQL Threat Intelligence Platform with GraphRAG

Deliverable	Review – I (10 marks)
Focus	Problem identification and design
Student	Shivansh
Programme	Computer Science / Information Technology
Institution	[College / University name]
Guide / Supervisor	[Name]
Roll number	[Roll no.]
Date	August 2026

Expected deliverables covered in this report

Problem statement
Literature survey (5–8 pages)
Gaps identified
Requirement analysis
Selection of NoSQL databases
Initial architecture
Graph / document schema
Project plan

Table of Contents

1. Abstract
2. Problem Statement
3. Literature Survey (5–8 pages)
    3.1 Cyber threat intelligence: definitions and the relationship problem
    3.2 Standards and sharing platforms
    3.3 Limitations of relational stores for CTI workloads
    3.4 NoSQL models and polyglot persistence
    3.5 Graph databases for cybersecurity knowledge
    3.6 Document stores for heterogeneous CTI reports
    3.7 Key–value caches and indicator volatility
    3.8 Vector databases and dense retrieval
    3.9 From RAG to GraphRAG
    3.10 Integrated CTI platforms
    3.11 Summary of the survey
4. Gaps Identified
5. Requirement Analysis
6. Selection of NoSQL Databases
7. Initial Architecture
8. Graph and Document Schema
9. Project Plan
    10. Conclusion of Review–I

References

Appendix A. Representative analyst questions

1. Abstract

Modern security operations generate overwhelming volumes of threat data whose real value lies not in isolated facts but in the relationships between attackers, campaigns, malware, and indicators of compromise. Traditional relational databases struggle with these highly connected, heterogeneous, and rapidly evolving datasets. This project designs a production-oriented Cyber Threat Intelligence (CTI) workstation that uses polyglot persistence: four complementary NoSQL data models, each assigned to the workload where it is operationally superior. A graph database (Neo4j) is the system of record for the threat knowledge graph and answers multi-hop relationship queries—such as shared attacker infrastructure—that would require expensive joins in SQL. A document store (MongoDB) is the system of record for raw, schema-variable threat reports, including STIX 2.1 bundles, without

rigid migrations. A key–value store (Redis) serves the enforcement-facing indicator blocklist with TTL-based automatic expiry, reflecting the short-lived nature of threat indicators. A vector database (Qdrant) indexes report embeddings for semantic similarity search. On top of these, a GraphRAG (Graph-augmented Retrieval-Augmented Generation) layer combines graph traversal, vector retrieval, and a locally hosted large language model so that analysts can ask natural-language questions and receive grounded, citable responses. An operations dashboard exposes evidence in real time—graph traversals, live TTL state, and semantic-similarity neighbourhoods—so that every answer can be audited against the stores that produced it. The result is a working CTI platform whose storage architecture is an engineering decision, not a single-database compromise.

Keywords: NoSQL, polyglot persistence, graph database, document store, key–value store, vector database, GraphRAG, cyber threat intelligence, Neo4j, MongoDB, Redis, Qdrant.

2. Problem Statement

2.1 Context

Security operations centres (SOCs) and CTI teams ingest reports, malware write-ups, indicator feeds, and adversary profiles that are simultaneously (i) highly connected, (ii) schema-unstable, (iii) time-decaying, and (iv) linguistically varied. The operational questions that matter are relational and semantic rather than tabular. Typical analyst questions include: which threat actors share command-and-control infrastructure; which campaigns reuse a commodity post-exploitation tool; which unstructured reports are narratively similar to a new spearphishing incident; and whether an indicator currently on a blocklist is still fresh. These questions cut across data shapes that a single storage engine handles poorly [1], [8], [9].

2.2 Statement of the problem

Existing CTI platforms do not close the loop that analysts actually need. Sharing systems such as MISP emphasise collection and distribution of events [5]. Knowledge-graph workstations such as OpenCTI model relationships but do not combine typed multi-hop graph retrieval, semantic report search, a volatile enforcement cache, and grounded natural-language Q&A in one control plane [16]. Language-model assistants, conversely, produce fluent text without typed graph constraints or live indicator state [21]. The operational problem is therefore: analysts cannot ask a multi-hop, evidence-backed question of their CTI estate and see, in one place, the graph path, the source report, the semantic neighbours, and whether an indicator is still actionable.

Formally: *design and implement an operational CTI platform that uses polyglot NoSQL persistence and GraphRAG so that relationship queries, variable reports, volatile indicators, and semantic search are each served by the data model best suited to that task, with an analyst interface that returns cited answers and an ingest path that keeps all four stores consistent.*

2.3 Why relational-only designs fail here

A normalised relational schema can represent actors, campaigns, and indicators, but multi-hop questions become recursive joins or closure tables whose cost grows with path length and with the heterogeneity of edge types [12]. Report bodies and nested, report-specific objects (ICS protocol lists, build-pipeline metadata, attribution confidence objects) force either sparse columns, Entity–Attribute–Value anti-patterns, or frequent migrations [8]. Indicator blocklists require aggressive expiry; relational rows do not expire themselves. Semantic similarity is not a native relational operator. Stonebraker and Çetintemel's argument that "one size fits all" has passed is therefore not a slogan in this domain—it is an observed mismatch between workload and engine [8].

2.4 Scope and non-goals

The project is a complete CTI workstation: ingest, persistence, GraphRAG query, analyst dashboard, and enforcement-facing blocklist. It is not a national CERT product and does not replace commercial telemetry crawlers; it is a real platform that an organisation can run on its own infrastructure. In scope: STIX-aligned knowledge graph, schema-variable report store, TTL blocklist for SOC/EDR consumers, vector search over report text, locally hosted LLM (no mandatory cloud API) so classified or TLP-AMBER text never leaves the perimeter, role-based access by TLP, and an ingest pipeline that fans out writes to all four stores. Out of scope: generating offensive payloads, scraping live victim networks, or claiming new cryptanalysis. Evaluation in later reviews will measure query correctness against gold graph paths, blocklist freshness, GraphRAG groundedness (citation coverage), and latency SLOs per store—not open-domain QA leaderboards.

2.5 Objectives

O1. To survey operational CTI systems, NoSQL models, RAG, and GraphRAG, and to state the engineering gap this platform closes.

O2. To select four NoSQL stores (graph, document, key–value, vector) with an explicit, testable rationale for each production workload.

O3. To specify a full architecture: ingest fan-out, GraphRAG orchestrator, TLP-aware API, and analyst operations dashboard.

O4. To define graph, document, key–value, and vector schemas that support multi-hop CTI queries and schema-variable reports.

O5. To plan implementation, evaluation (correctness, latency, groundedness), and subsequent project reviews.

3. Literature Survey

This survey is organised around the data problems of operational cyber threat intelligence rather than around product catalogues. It first establishes what CTI is and why relationships dominate its value; reviews standards and sharing platforms; then examines each candidate data model (relational, graph, document, key–value, vector) and the generation stack (RAG, GraphRAG). It closes by positioning integrated platforms such as MISP and OpenCTI, which is the immediate background for the gaps in Section 4.

3.1 Cyber threat intelligence: definitions and the relationship problem

Tounsi and Rais distinguish strategic, operational, and technical threat intelligence, and argue that the last of these—indicators, malware artefacts, and tactical procedures—is what automated defences actually consume [1]. Skopik, Settanni, and Fiedler frame sharing itself as a collective-defence problem: a problem shared is only "halved" if the receiving organisation can correlate the shared object with local context [2]. Both surveys converge on a point that storage architecture must respect: an IP address, a malware family name, or a spearphishing lure is weakly informative in isolation. Value appears when the same infrastructure is reused across actors, when a tool is common to crime and espionage clusters, or when a campaign's initial-access technique matches a new report's narrative. Liao et al. showed that even open-source "IOC games" are really extraction-plus-correlation problems, not lookup problems [6]. Husari et al.'s TTPDrill likewise treats unstructured CTI text as a source of ATT&CK-like threat actions rather than as a bag of tokens [7]. Any storage architecture that cannot cheaply ask "what else is connected, two or three hops away?" is therefore fighting the structure of the domain.

Industry practice agrees. MITRE ATT&CK organises adversary behaviour as a graph of tactics and techniques rather than as a flat signature list [4]. ENISA threat-landscape reporting and vendor M-Trends-style analyses repeatedly describe clustering of activity by shared tooling and infrastructure, not by a single primary key [27], [28]. The implication for this project is direct: the canonical CTI query is a path query over a heterogeneous graph, optionally grounded in long-form text.

3.2 Standards and sharing platforms

OASIS STIX 2.1 models CTI as a graph of STIX Domain Objects (SDOs) and STIX Relationship Objects (SROs), exchanged over TAXII [3]. The standard is explicitly relationship-centric: an indicator "indicates" a malware family; an intrusion set "uses" a tool; a campaign "targets" an

identity. That is a property-graph schema in all but product name. Wagner et al. describe MISP as a collaborative sharing platform whose core abstraction is the event—an extensible bundle of attributes, objects, and galaxies [5]. MISP's historical persistence engine is relational (MySQL/MariaDB), with objects mapped into tables and JSON-ish blobs; the sharing UX is excellent, but multi-hop "who shares this C2 with whom?" is not the native query model. OpenCTI, by contrast, is built around a knowledge-graph mental model and a modern microservice stack (search, queue, object storage) [16]. It is the closest production analogue to the present work. It still leaves three operational holes this platform fills: a first-class TTL enforcement cache distinct from historical intelligence; a vector index over report narrative for semantic recall; and a GraphRAG control plane that cites graph edges, document identifiers, and live IOC state together rather than offering search and chat as disconnected features.

The standards landscape therefore supplies the ontology (ATT&CK, STIX) and the sharing UX (MISP, OpenCTI, commercial TI platforms), but it does not settle the storage question. STIX can be serialised as JSON into a document store, projected as nodes and edges into Neo4j, indexed as embeddings in a vector database, or cached as indicator keys in Redis. The interesting design problem —the one this project takes as primary—is which projection to use for which operation.

3.3 Limitations of relational stores for CTI workloads

Cattell's survey of scalable SQL and NoSQL stores remains a useful map of the design space: clustered relational systems scale some workloads, but they do not erase the impedance mismatch between tables and variable documents or recursive graphs [10]. Stonebraker's "one size fits all" critique is older than the NoSQL marketing wave and more precise than it: specialised engines win when the data model and the query operators match the workload [8]. CTI exhibits three such mismatches at once.

First, graph-shaped queries. Computing "actors two hops from FIN7 via shared tooling" in SQL is a join of actor–uses–malware with malware–used-by–actor, unioned with further hops, with DISTINCT to break cycles. Query planners do this; they do not like doing it repeatedly at interactive latency on dense hubs such as Mimikatz or Cobalt Strike. Graph databases treat that pattern as a first-class traversal [12], [26].

Second, schema drift. A SolarWinds report needs a nested build-pipeline object and a list of MITRE IDs; an Industroyer report needs ICS protocol names and outage duration; a POS-skimming report needs payment brands. Forcing these into one wide table produces nulls and migrations; splitting them into subclass tables produces join storms on the common case (list all reports mentioning an actor). Document stores accept this heterogeneity as the default [9], [17].

Third, volatility. Many indicators are useful for hours to days, not years. Relational delete jobs can implement expiry; they do not make expiry a property of the datum. Key–value stores with TTL do [18]. Brewer's CAP discussion applies when stores are clustered independently: this architecture picks consistency and lifetime semantics per working set—strong, typed consistency on the graph and documents; eventual expiry on the hot blocklist—rather than one global rule [11].

3.4 NoSQL models and polyglot persistence

Sadalage and Fowler popularised polyglot persistence: using different data stores for different kinds of data inside one application, rather than stretching a single engine [9]. This project treats that as an operational requirement of CTI, not a slogan. Four models are in scope.

Property graphs. Nodes have labels and properties; relationships are typed, directed, and can themselves carry properties. Cypher (Neo4j) and similar languages express variable-length paths without encoding the path as a table [12], [26]. This is the natural home of STIX-like CTI.

Documents. JSON/BSON documents are trees. They fit reports, after-action write-ups, and vendor PDFs once parsed. Schema-on-read means two reports in the same collection need not share nested keys [17].

Key–value. The simplest NoSQL model. A key (for example ioc:ip:185.86.77.12) maps to a blob. Operations are O(1) GET/SET with optional expiry. The model is a poor fit for "find all actors related to this campaign" and an excellent fit for "is this indicator currently blocked?" [18].

Vectors. Embeddings place passages in a high-dimensional space so that cosine or inner-product nearest neighbours approximate semantic similarity. Indexes such as HNSW make approximate k-NN practical [19], [20], [30]. This is not a replacement for a knowledge graph: two reports can be semantically close without sharing a single infrastructure node, and two actors can share infrastructure without similar prose.

Polyglot persistence is justified only when the application actually issues the queries that each engine is good at. A system that writes everything to MongoDB and then pretends to be a graph is not polyglot; it is a document store with extra latency. The architecture in Section 7 therefore assigns each engine a closed set of operations and forbids using it as a general dumping ground.

3.5 Graph databases for cybersecurity knowledge

Robinson, Webber, and Eifrem argue that graphs win when the problem is the connections [12]. Cybersecurity was an early adopter. Noel et al.'s CyGraph combined network topology, vulnerabilities, and attack paths into a graph analytic environment, showing that path queries support

both defence and mission impact [13]. Piplai et al. constructed cybersecurity knowledge graphs from malware after-action reports, linking extracted entities into a typed graph that can be queried beyond keyword search [14]. Sarhan and Spruit's Open-CyKG similarly publishes an open CTI knowledge graph extracted from reports [15]. These works establish two results this project relies on: (1) CTI entities and relations can be extracted or curated into a property graph; (2) the graph is useful for questions that keyword search misses.

They also leave a storage lesson unstated. Once the graph exists, keeping the raw report only as a node property does not scale to long, variably structured documents, and it does not give semantic nearest-neighbour search. Conversely, a knowledge graph without a vector index cannot retrieve "reports like this spearphish against a political organisation" unless that phrase was pre-encoded as a node. Graph-plus-document-plus-vector is therefore a completion of the CyKG line of work, not a contradiction of it.

Neo4j is selected as the graph engine because the labelled-property-graph model maps onto STIX-like CTI, Cypher expresses variable-length paths as first-class queries, and the engine is proven in production knowledge-graph deployments [12], [26]. Alternatives (JanusGraph, Memgraph, Amazon Neptune, Kuzu) do not change the data-model argument; clustering Neo4j is a deployment choice, not a schema change.

3.6 Document stores for heterogeneous CTI reports

Technical CTI is not only indicators. It is prose: how SUNBURST waited through a dormant period; how NotPetya's ransom note was a façade; how Scattered Spider social-engineers help desks. Liao et al. and Husari et al. both start from unstructured text [6], [7]. Piplai et al. start from after-action reports [14]. A document database is the appropriate system of record for those artefacts because each report is a self-contained JSON tree whose nested shape follows the incident, not a global schema [9], [17].

MongoDB is selected as the report system of record: BSON, secondary indexes on common fields (actor id, TLP, date), change streams for ingest fan-out, and the ability to store attribution objects, YARA rule names, ICS protocol arrays, and missing IOC lists side by side. Couchbase, DocumentDB, or PostgreSQL JSONB could hold the same trees; they are not chosen because the document model is required, and MongoDB's operational tooling matches a CTI ingest pipeline.

3.7 Key–value caches and indicator volatility

Indicators of compromise have a half-life. Fast-flux infrastructure, sinkholed domains, and rotating leak-site hostnames mean that yesterday's blocklist entry may be actively harmful (false positives) or

simply dead. Sharing platforms retain history—which is necessary for forensics—but enforcement paths need a hot set. Redis-style stores provide GET/SET with per-key TTL and eviction policies [18]. In a SOC pipeline this is the difference between a SIEM table scan and an in-memory membership check on a firewall or EDR integration.

Redis is not "a cache because web applications have caches." It is the enforcement plane: freshness as a first-class property of each indicator [18]. The operations dashboard shows remaining TTL because SOC operators must know whether a block is still live. When a key expires, the graph and the report still retain historical presence; the blocklist no longer asserts that the indicator is currently actionable. Downstream firewall or EDR integrations consume this hot set, not a SQL table scan.

3.8 Vector databases and dense retrieval

Karpukhin et al.'s Dense Passage Retrieval showed that dual-encoder embeddings outperform BM25 on many open-domain QA retrieval tasks [22]. Malkov and Yashunin's HNSW graphs made approximate nearest neighbour search efficient enough for interactive applications [19]. FAISS demonstrated billion-scale similarity search [30]. Productised vector databases such as Qdrant wrap these indexes with payload filters and collections [20]. For CTI, the payoff is semantic recall: a query about "supply-chain compromise of signed software updates" should retrieve SolarWinds, MeDoc/NotPetya, and 3CX write-ups even if they share few keywords. That is precisely the class of match that graph structure alone does not give (those campaigns do not share a single actor) and that Boolean keyword search on "supply chain" only partially gives.

Vector search fails in characteristic ways that GraphRAG must handle in production. Commodity terms ("phishing", "Mimikatz") retrieve too much. Rare malware family names retrieve well only if they appeared in the embedding corpus. The platform therefore uses vectors to pull narrative context, then constrains and expands with the graph so that semantic recall cannot invent a relationship that Cypher does not support [23], [24].

3.9 From RAG to GraphRAG

Lewis et al. introduced retrieval-augmented generation as a way to ground sequence-to-sequence models in a non-parametric memory, reducing hallucination on knowledge-intensive tasks [21]. The RAG pattern—retrieve passages, then generate—has become the default architecture for "chat with your documents." It is not sufficient for CTI. Passages do not reliably encode multi-hop constraints such as "shared C2 but different malware families." Yasunaga et al.'s QA-GNN already combined language models with knowledge-graph reasoning for QA [25]. Edge et al.'s Graph RAG (Microsoft) builds a graph over source documents (including community summaries) so that query-focused

summarisation can use both local neighbours and global structure [23]. Peng et al. survey the rapidly growing GraphRAG family: retrieve nodes/paths/subgraphs, optionally rerank, then generate [24].

This platform instantiates a CTI-specific GraphRAG loop rather than Microsoft's community-summary construction over general corpora. Retrieval is four-channel: (1) embedding nearest neighbours in Qdrant; (2) Cypher expansions in Neo4j; (3) full-document hydration in MongoDB; (4) live IOC state in Redis. Generation is constrained to that context pack and is executed on a locally hosted LLM so that TLP-restricted reports never require a cloud API. Relative to Lewis et al. the contribution is the typed graph channel plus live blocklist state; relative to Edge et al. it is the polyglot CTI mapping and citation into operational stores; relative to OpenCTI it is grounded natural-language Q&A that is obligated to name its graph edges, document ids, and IOC TTLs.

Risks are acknowledged. LLMs still paraphrase incorrectly even with retrieved context. The mitigation is architectural and operational: every answer is displayed beside the retrieved hops, passages, and IOC states; the model is a renderer of evidence, not an authority; and later evaluation scores answers against gold paths in the knowledge graph and against a citation-coverage rubric.

3.10 Integrated CTI platforms: what exists and what is still missing

MISP remains the open standard for sharing indicators and events among CSIRTs [5]. OpenCTI is the leading open graph-shaped CTI workstation [16]. Commercial platforms (Recorded Future, ThreatConnect, Mandiant Advantage, and others) add scoring, crawling, and analyst workflow [28]. Academic CyKGs add extraction from reports into graphs [14], [15]. RAG systems add NL interfaces over text dumps [21]. No widely deployed system simultaneously (a) keeps relationship, document, volatile-indicator, and semantic-search workloads on native NoSQL engines with an explicit operation set per store, (b) runs GraphRAG over CTI with citations into those engines, and (c) exposes hops, TTLs, and embedding neighbourhoods so that an analyst can audit why an answer was produced. That operational combination is the niche of this platform. It is designed to ingest a growing corpus (STIX bundles and free-text reports) rather than to remain a static sample, and to be operated as a workstation rather than as a single-database application.

3.11 Summary of the survey

The literature yields five design commitments. CTI value is in relationships and in prose, not in isolated indicators [1], [6]. Standards already describe a graph (STIX, ATT&CK) [3], [4]. One storage engine cannot serve path queries, schema-variable documents, volatile hot keys, and semantic k-NN equally well [8], [9]. Knowledge graphs for cyber exist but rarely close the loop with vectors and TTL caches [13]–[16]. RAG must be graph-augmented to answer multi-hop CTI questions without

unconstrained hallucination [21], [23], [24]. Section 4 turns these commitments into explicit gaps; Sections 6–8 turn them into store selection, architecture, and schemas.

4. Gaps Identified

The survey does not claim that CTI platforms or GraphRAG papers are absent. It claims that the following gaps remain, and that they are the ones this project is designed to close.

Gap G1 — One engine is forced to serve four incompatible CTI workloads

Production stacks often hide polyglot choices behind a single API, or collapse everything into search-plus-relational. Path queries, schema-variable reports, volatile hot keys, and dense k-NN then share an engine that is optimal for at most one of them [8], [9]. The result is either slow multi-hop correlation or a second unofficial store that operators cannot query together.

Gap G2 — Sharing platforms are not query-native graphs with enforcement state

MISP's strength is dissemination of events, not interactive multi-hop queries over a labelled property graph coupled to a live blocklist [5]. OpenCTI is graph-native as a product but does not treat TTL enforcement, vector narrative search, and cited GraphRAG as first-class, co-equal planes [16].

Gap G3 — Knowledge graphs stop before volatile indicators and semantic recall

Cybersecurity knowledge graphs extract entities from reports [14], [15] but seldom pair the graph with (a) a TTL blocklist that can disagree with historical fact—necessary for SOC enforcement—and (b) a vector index over the same report text. Those two absences are exactly where Redis and Qdrant belong in a real workstation.

Gap G4 — RAG over CTI is under-constrained

Vanilla RAG retrieves similar paragraphs and lets the LLM narrate [21]. Multi-hop CTI questions ("shared C2 with APT28") are graph questions. GraphRAG work is recent and mostly evaluated on general or enterprise corpora [23], [24], not on ATT&CK-aligned CTI with citable report identifiers and live IOC state that a SOC can act on.

Gap G5 — Answers are not auditable against the stores that produced them

Analysts cannot currently see, for a single question, which hops were traversed, which report chunks were similar, which documents were hydrated, and whether named IOCs are still in the hot blocklist. Without that observability, GraphRAG is an un-auditable chatbot. The operations dashboard is therefore part of the control plane: every answer must be replayable against the stores that produced it.

Table 1 maps each gap to a project response.

Gap	Observation from literature	Project response
G1	One store forced to serve path, document, TTL, and k-NN	Four native engines; closed operation set per store
G2	MISP/OpenCTI lack GraphRAG + live TTL plane	STIX-like graph + Redis enforcement cache + cited NL Q&A
G3	CyKGs lack TTL cache and vectors	Redis blocklist + Qdrant chunks beside Neo4j
G4	RAG is passage-only; GraphRAG rarely CTI-specific	Four-channel GraphRAG + local LLM + citations
G5	Answers cannot be audited to source stores	Traversal, TTL, similarity, and document replay in the UI

Table 1. Gaps and corresponding design responses.

5. Requirement Analysis

5.1 Stakeholders

- SOC / CTI analyst (primary): relationship and narrative questions, evidence inspection, TLP-aware report access.
- CTI lead / detection engineer: publishes indicators to the hot blocklist, sets TTLs, reviews GraphRAG citations before pushing detections.
- Platform operator / SRE: store health, ingest jobs, model endpoint, backup of graph and documents.
- Downstream consumer (firewall, EDR, SIEM): O(1) membership checks against the Redis blocklist only.

5.2 Functional requirements

ID	Name	Description
FR1	Ingest	Accept STIX 2.1 bundles and free-text CTI reports with non-uniform nested fields; fan-out to graph, document, vector, and (where applicable) blocklist stores.
FR2	Graph query	Answer multi-hop questions (shared infrastructure, shared tooling, common victims) as path queries over the knowledge graph.
FR3	Blocklist	O(1) indicator lookup with per-key TTL; expired keys leave the hot set while remaining in graph/history; exportable to EDR/firewall consumers.
FR4	Semantic search	Retrieve report chunks by embedding similarity, with payload filters (actor, TLP, date).
FR5	GraphRAG Q&A	Natural-language question → retrieve from all four stores → generate a cited answer on a locally hosted LLM.
FR6	Hydration	Open the full MongoDB document for a graph Report node or a vector hit, subject to TLP policy.
FR7	Evidence UI	Show traversals, remaining TTL, and similarity neighbourhoods so every answer is

		auditable.
FR8	Corpus	Load an initial public-knowledge CTI graph and reports; support ongoing ingest of additional STIX/report documents.
FR9	Health / SLO	Report liveness of Neo4j, MongoDB, Redis, Qdrant, and the LLM; record per-store latency on GraphRAG calls.
FR10	Replay / audit	Persist which hops, chunks, documents, and IOCs were used for each GraphRAG answer (audit log).
FR11	Access control	Authenticate analysts; enforce TLP (WHITE/GREEN/AMBER/RED) on document bodies and on generated answers.
FR12	Blocklist publish	Authorised role can add/revoke an IOC with a chosen TTL without rewriting the historical graph.

Table 2. Functional requirements.

5.3 Non-functional requirements

ID	Name	Description
NFR1	Data residency	LLM and CTI data remain on organisation-controlled infrastructure; no mandatory cloud API.
NFR2	Interactive latency	Redis GET p99 < 5 ms; graph 1–3 hop p95 < 500 ms on the working corpus; GraphRAG end-to-end p95 < 15 s including LLM.
NFR3	Store isolation	A Redis miss must not be "fixed" by scanning MongoDB on the enforcement path; each store's contract is closed.
NFR4	Groundedness	Every generated sentence must be tie-able to a graph edge, chunk id, or report id; otherwise the system refuses.
NFR5	Reproducibility	Deterministic ingest; compose-based deployment; versioned schemas.
NFR6	Ethics / safety	No exploit generation; no live targeting; public or operator-supplied indicators only.
NFR7	Usability	Analyst can complete UC1–UC4 without leaving the dashboard; evidence panel always visible beside the answer.
NFR8	Availability	Read path (graph, documents, search, Q&A) degrades gracefully if Redis or LLM is down; enforcement path fails closed if Redis is down.

Table 3. Non-functional requirements.

5.4 Use cases (brief)

UC1 Shared infrastructure. Analyst asks which actors share C2 with APT28. System traverses USES_INFRA, returns Sandworm (shared VPS) and any Beacon overlap, cites reports, shows live Redis state of those IPs/domains.

UC2 Semantic analogue. Analyst asks for activity similar to spearphishing against political organisations. Qdrant returns DNC-related chunks; graph expands to APT28/APT29; MongoDB hydrates full reports.

UC3 Commodity-tool caution. Analyst asks who uses Mimikatz. Graph returns many actors; the answer must state that the tool is a weak attribution feature and must pair it with victim sector or C2 context [6].

UC4 Expiry. Dashboard shows an IOC TTL reach zero; blocklist miss; graph still holds historical HAS_IOC/OBSERVED_AS.

5.5 Success criteria for Review–I

Review–I succeeds if the operational problem is sharp, gaps versus MISP/OpenCTI/RAG are cited, four stores are justified as production choices, architecture includes ingest and access control, and schemas are specific enough to implement. Implementation belongs to Review–II.

6. Selection of NoSQL Databases

Selection follows a single rule: pick the model whose native operators match the CTI operation, then pick a production-grade engine that implements that model. Table 4 is the decision record. Deployment may start as four containers on one host and scale each engine independently later; the schema does not change.

Workload	Native operators needed	Rejected alternatives	Selected engine
Multi-hop CTI relations	Variable-length path, typed edges	SQL recursive CTE (verbose, costly); document $graphLookup (not first-class)	Neo4j 5 (labelled property graph, Cypher)
Variable CTI reports	Schema-on-read JSON trees, index common fields	Wide SQL table; XML store; graph properties for long prose	MongoDB 7 (documents / BSON)
Hot IOC blocklist	O(1) GET + per-key TTL	SQL DELETE job; Mongo TTL index (heavier); graph nodes (no native countdown UX)	Redis 7 (key–value + EXPIRE)
Semantic report search	k-NN over dense vectors, payload filters	BM25-only; storing vectors in Postgres without HNSW ops; graph full-text	Qdrant (HNSW, cosine)

Table 4. NoSQL selection rationale (model first, product second).

6.1 Neo4j — graph

Neo4j implements the labelled property graph that STIX-like CTI already assumes [3], [12], [26]. Cypher MATCH (a:ThreatActor)-[:USES_INFRA]->(i)<-[:USES_INFRA]-(b) is the shared-infrastructure query that would be a self-join puzzle in SQL. Relationship types (USES,

ATTRIBUTED_TO, HOSTS, MENTIONED_IN) are data, not column names, so adding SHARES_UNIT between GRU clusters does not require a migration. Neo4j is rejected as a store for full report prose (property size and query mix) and as a blocklist (no TTL semantics comparable to Redis).

6.2 MongoDB — document

MongoDB holds the system of record for reports. A single reports collection stores documents that share a small core (_id, title, source, date, tlp, body, actors[]) and diverge on nested keys (attribution, build_pipeline, ics_protocols, yara_rule_name). That is the textbook document-model win [9], [17]. MongoDB is rejected for multi-hop paths and for sub-millisecond membership tests.

6.3 Redis — key–value

Each hot indicator is a key ioc:{type}:{value} with a JSON payload and EXPIRE. TTL is the reason for the engine, not an afterthought [18]. Historical presence remains in Neo4j/MongoDB. Redis is rejected as a knowledge graph (no typed multi-hop) and as a report store (blobs without rich query).

6.4 Qdrant — vector

Report text is chunked, embedded, and indexed under cosine distance with HNSW [19], [20]. Payloads keep report_id so hits can hydrate MongoDB and pin graph Report nodes. Qdrant is rejected as a source of truth for relationships (nearest neighbour ≠ “shares C2”).

6.5 What was not selected, and why

Cassandra / wide-column: excellent for append-heavy telemetry time series, not for CTI path queries or variable report trees. Elasticsearch as the only store: strong full-text, weaker as a typed graph and as a TTL blocklist with per-key expiry semantics. PostgreSQL + JSONB + pgvector + recursive CTE can approximate all four and is a valid industrial compromise for a small team that wants one operator surface; it is rejected here because the four workloads have incompatible SLOs (sub-5 ms membership vs multi-hop graph vs k-NN vs schema-on-read documents), and collapsing them would recreate the “one size fits all” failure [8]. Cloud-only embedding APIs were rejected under NFR1.

7. Initial Architecture

The architecture has four layers (Figure 1). Ingest accepts STIX 2.1 bundles and free-text reports, extracts entities, and fans out writes: graph merges, document upsert, chunk embed/upsert, and optional Redis SET with EXPIRE for newly actionable IOCs. The presentation layer is an analyst operations console (graph explorer, report browser, live blocklist, semantic search, GraphRAG). The

orchestration layer is the GraphRAG service plus TLP-aware API and audit log. The persistence layer is the four NoSQL engines. There is no fifth catch-all database.

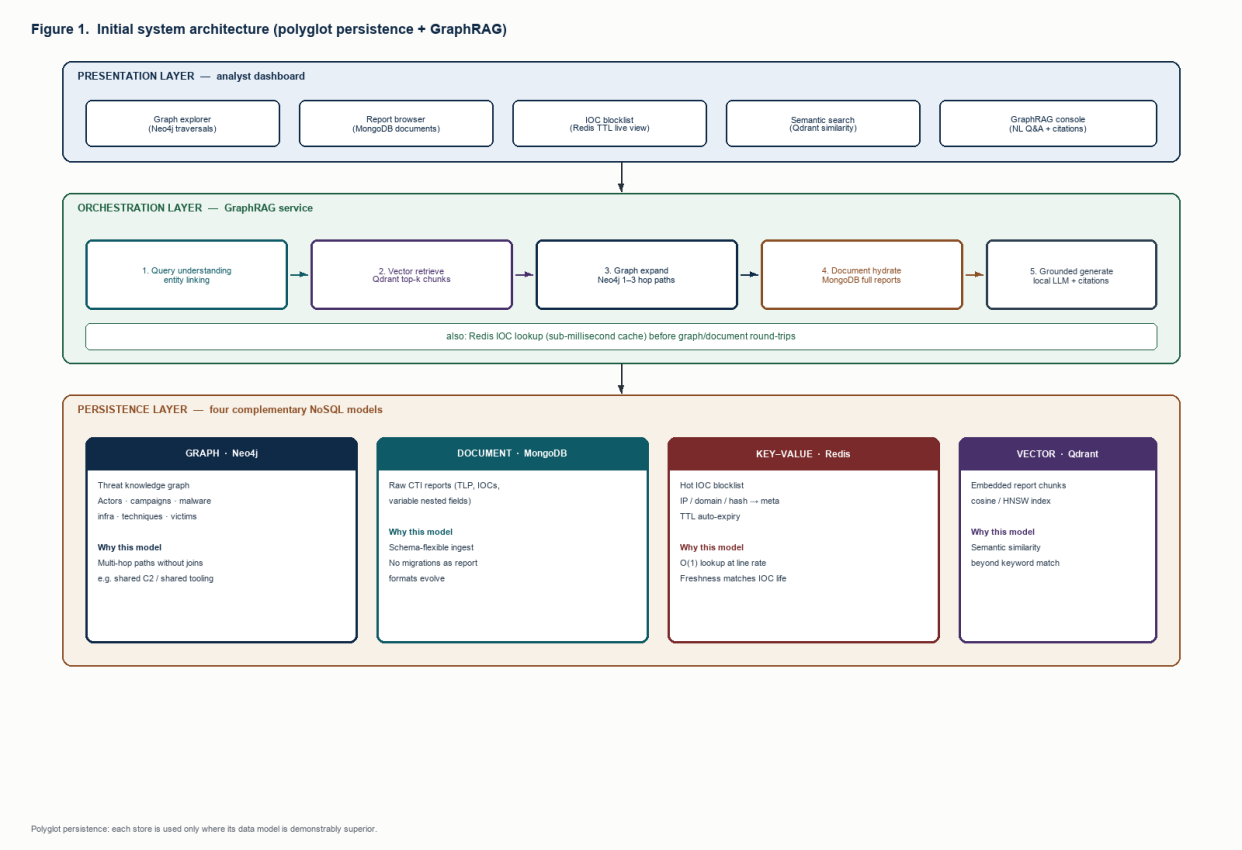


Figure 1. Initial system architecture: presentation, GraphRAG orchestration, and four NoSQL stores.

7.1 GraphRAG control flow

A question such as "Which actors share C2 infrastructure with APT28, and which reports discuss that overlap?" is executed as follows. Entities in the question are linked to graph ids (APT28 → ThreatActor apt28). Qdrant embeds the question and returns top-k chunks. Neo4j expands 1–3 hops from linked entities, biased toward USES_INFRA, HOSTS, and MENTIONED_IN. MongoDB loads full reports for the union of chunk report_ids and graph Report nodes. Redis returns TTL and severity for any indicator values mentioned in the question or the retrieved subgraph. The context pack—graph facts, passages, report ids, IOC states—is sent to a locally hosted LLM with an instruction to refuse unsupported claims. The dashboard then replays the same artefacts: hops animate, similar chunks plot, TTLs tick.

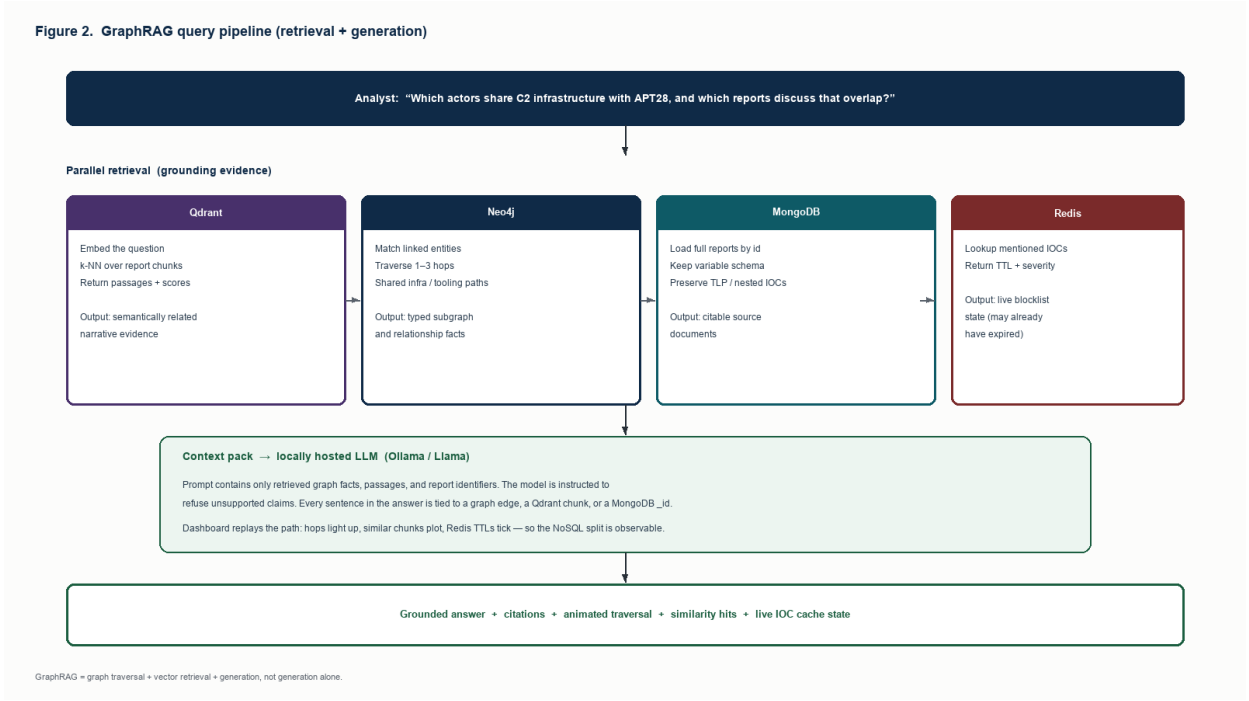


Figure 2. GraphRAG pipeline: parallel retrieval from four stores, then grounded generation.

7.2 Component responsibilities

Component	Responsibility
Ingest pipeline	STIX/report parse; entity link; fan-out to Neo4j, MongoDB, Qdrant, Redis; idempotent upserts
Analyst console	Graph canvas; report viewer; TTL blocklist; similarity map; GraphRAG with evidence panel
GraphRAG service	Entity linking, retrieval planning, prompt assembly, LLM call, citation packing
API + policy	AuthN/AuthZ; TLP filtering; audit log of queries and answers
Neo4j	Canonical relationship store; path queries; subgraph for the console
MongoDB	Canonical report/STIX store; variable schema; citation targets
Redis	Hot blocklist for enforcement; TTL; not a historical archive
Qdrant	Chunk vectors; k-NN; payload → report_id
Local LLM	Render evidence into prose; no unaudited write path to stores

Table 5. Component responsibilities.

7.3 Data movement and keys

Cross-store references are explicit and minimal. Report.id in Neo4j equals MongoDB _id. Indicator.value (and type) in the graph equals the Redis key suffix and may appear in MongoDB iocs[]. Qdrant payload.report_id equals MongoDB _id. Ingest is the consistency boundary: a single ingest job writes Neo4j, MongoDB, Qdrant, and optionally Redis in a defined order, records a job id, and is idempotent on retry. Runtime GraphRAG is read-mostly. There is no distributed two-phase

commit; a failed fan-out is retried from the ingest log. That is the production pattern for polyglot CTI, not a shortcut.

7.4 Security, TLP, and ethics

The LLM is local so that TLP-AMBER/RED text never depends on a third-party API (NFR1). Analysts authenticate; document bodies and GraphRAG context packs are filtered by TLP (FR11). The audit log stores question, retrieved ids, and answer for later review. The corpus may begin with public-knowledge campaigns and operator-supplied indicators; the system is a reader of threat descriptions, not a generator of offensive capability. Downstream blocklist consumers receive only Redis keys, never full AMBER report bodies.

8. Graph and Document Schema

8.1 Graph schema (Neo4j)

Figure 3 shows the labelled property graph. Node labels follow STIX-like CTI nouns. Relationship types are verbs that Cypher can match directly. Commodity tools are permitted to become hubs; the schema does not pretend that Mimikatz is actor-unique, because the literature says it is not [6].

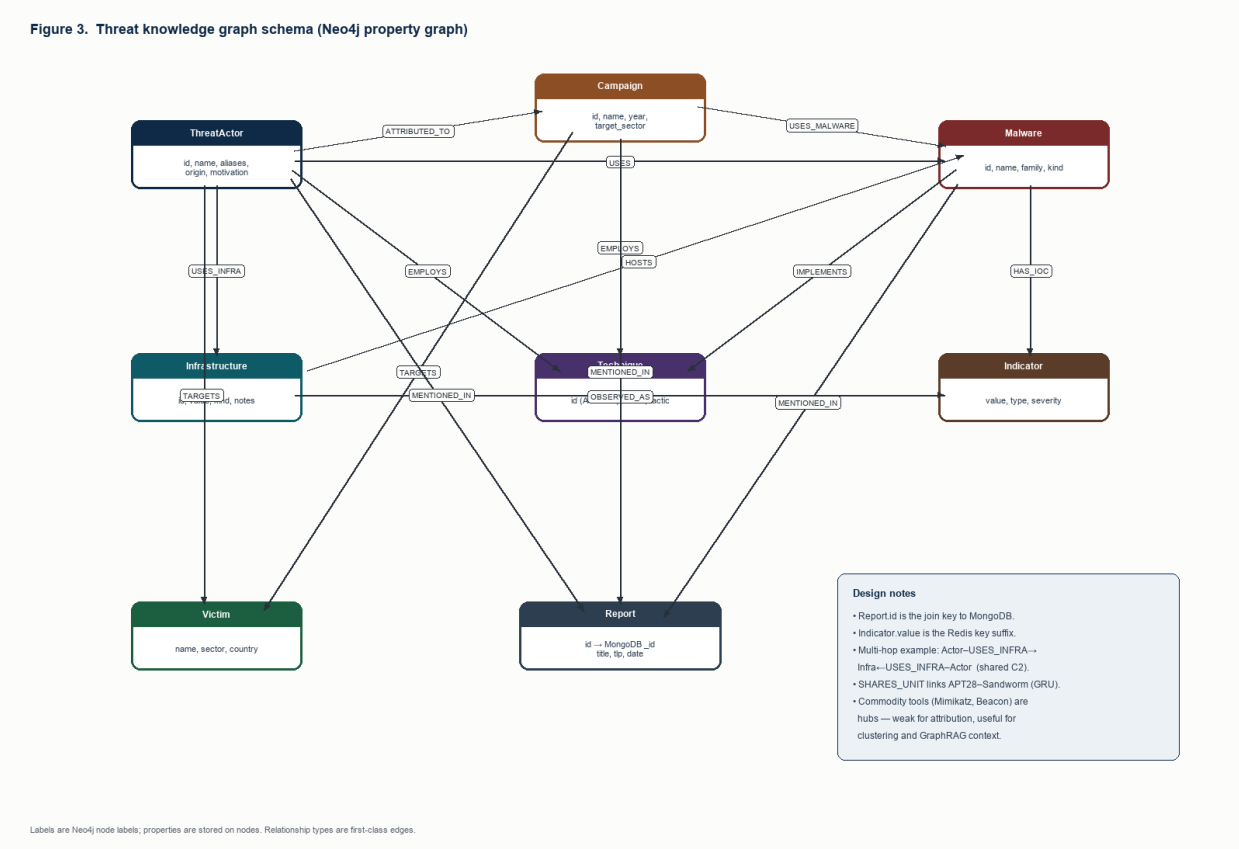


Figure 3. Threat knowledge graph schema (nodes, properties, and relationship types).

8.1.1 Node labels and properties

Label	Properties (core)
ThreatActor	id, name, aliases[], origin, sponsor, motivation, sophistication
Campaign	id, name, year, target_sector, summary
Malware	id, name, family, kind
Infrastructure	id, value, kind (c2-domain | c2-ip | …), notes
Technique	id (ATT&CK), name, tactic
Indicator	value, type (ip|domain|hash), severity
Victim	id, name, sector, country
Report	id (MongoDB _id), title, source, date, tlp

Table 6. Graph node labels.

8.1.2 Relationship types

Type	Endpoints
ATTRIBUTED_TO_CAMPAIGN	ThreatActor → Campaign
USES / USES_MALWARE	Actor/Campaign → Malware
USES_INFRA	ThreatActor → Infrastructure
HOSTS	Infrastructure → Malware
EMPLOYS / IMPLEMENTS	Actor/Campaign/Malware → Technique
HAS_IOC / OBSERVED_AS	Malware/Infrastructure → Indicator
TARGETS	Actor/Campaign → Victim
MENTIONED_IN	Actor/Campaign/Malware → Report
SHARES_UNIT	ThreatActor → ThreatActor (e.g. GRU clusters)

Table 7. Graph relationship types.

8.1.3 Representative Cypher (design-level)

Shared infrastructure, the flagship multi-hop query, is conceptually: match actors who USES_INFRA the same Infrastructure node as APT28, returning the node, the kind of infrastructure, and reports MENTIONED_IN along the path. Shared tooling is the same pattern on Malware hubs (Cobalt Strike, Mimikatz) with an explicit caution in the GraphRAG prompt that hub nodes are weak attribution. These queries are the reason Neo4j was selected in Section 6; they are not implementable as single-table selects.

8.2 Document schema (MongoDB)

The reports collection is schema-flexible by design (Figure 4). A core envelope is conventional so that the UI can list documents. Nested objects are allowed to appear or vanish per report. That variation is the proof of the document model: the same collection holds a GRU spearphish report with

an attribution object, a SolarWinds report with a build_pipeline object, and an ICS report with ics_protocols and no iocs[] array at all.

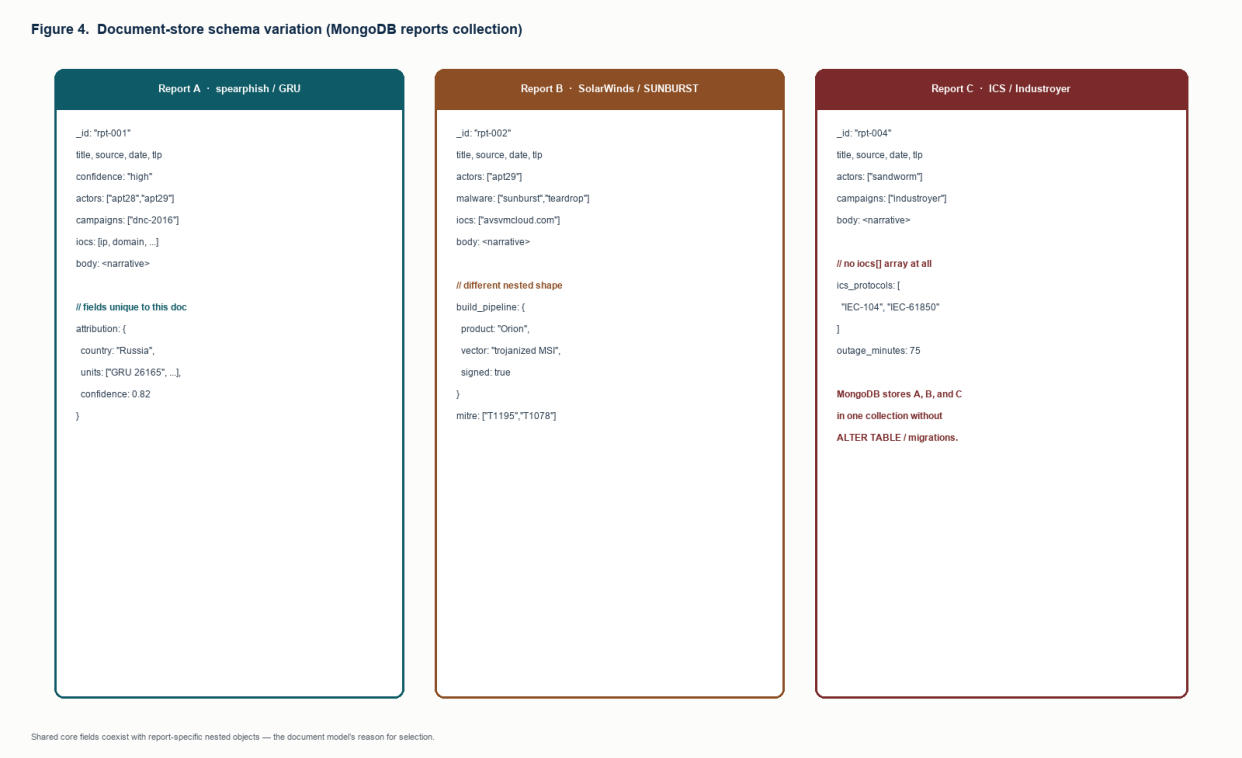


Figure 4. Schema variation across three reports in one MongoDB collection.

8.2.1 Core envelope (present on most documents)

- _id string — same as graph Report.id
- title, source, date, tlp, confidence, body
- actors[], campaigns[], malware[], iocs[] — optional arrays of ids/values

8.2.2 Optional, report-specific fields (examples)

- attribution { country, units[], confidence }
- build_pipeline { product, vector, signed }
- ics_protocols[], outage_minutes
- mitre[], cve[], wormable, kill_switch
- payment_brands[], lure_themes[], initial_access, idp[]
- yara_rule_name, package { name, platforms[] }

Indexes (design): _id (primary), actors, date, tlp. No unique constraint on nested optional keys.

8.3 Redis key schema

Key: ioc:{type}:{value} → JSON { value, type, actor, campaign, severity, ttl, published_by, published_at }. Expiry: EXPIRE set to the operational TTL chosen by the detection engineer (hours to days, not seconds). Secondary index: sorted set ioc:index scored by expiry unix timestamp, used

to list the hot set in the console and to drive expiry alerts. Redis is not queried by actor name; that remains a graph query. Enforcement consumers use EXISTS/GET only.

8.4 Qdrant collection schema

Collection threat_chunks. Vector: dense embedding of chunk text (dimension fixed at seed time). Distance: cosine. Payload: { id, report_id, title, source, date, chunk_index, text, actors[], campaigns[] }. Point id is a monotonic chunk id. Filter examples: actors contains apt28. k-NN is always over vectors; filters only restrict the candidate set [20].

8.5 Cross-model integrity rules

- Every MongoDB _id has a Report node; every Report node has a document.
- Every Redis key's value is allowed to exist as an Indicator and/or as a string in some document's iocs[].
- Every Qdrant point.report_id exists in MongoDB.
- Deleting a report in a later ingest version must delete its chunks and detach MENTIONED_IN edges (implementation Review–II).

9. Project Plan

The plan runs from Review–I (this design) through a working workstation and evaluation. Weeks are for a major-project calendar. Implementation is of the real architecture in Sections 7–8, including ingest and TLP, not a reduced mock.

Week	Milestone	Work
1–2	Review–I	Problem, survey, gaps, requirements, store selection, architecture, schemas
3–4	Platform foundation	Four stores, health/SLO endpoints, ingest contract, auth stub
5–6	Graph + documents	Neo4j schema + Cypher library; MongoDB reports/STIX + TLP-aware API
7	Enforcement plane	Redis IOC keys, TTL publish/revoke, consumer GET path, console
8	Vectors	Chunking, embeddings, Qdrant upsert, filtered similarity search
9–10	GraphRAG + audit	Orchestrator, local LLM, citations, replay log
11	Analyst console	Five surfaces; UC1–UC4; ingest UI for new reports
12	Review–II	Working workstation; implementation report
13–14	Evaluation	Gold-path correctness; latency SLOs; citation coverage; ingest consistency tests
15–16	Review–III	Write-up, limitations, operational runbook

Table 8. Project plan mapped to reviews.

9.1 Risks and mitigations

Risk	Mitigation
Local LLM too slow / unavailable	Retrieve-only mode still returns graph, chunks, documents, IOCs; LLM optional for prose
Ingest fan-out partial failure	Job log + idempotent retry; GraphRAG refuses docs without a Report node
Corpus growth / hub nodes	Query budgets (max hops, max neighbours on Mimikatz/Beacon); payload filters in Qdrant
Hallucinated GraphRAG answers	Refuse-if-missing prompt; evidence panel mandatory; citation-coverage metric
Scope creep (clone all of OpenCTI)	Freeze FR1–FR12; TAXII pull is ingest, not a full sharing community

Table 9. Risks and mitigations.

9.2 Review mapping

Review–I (this document): problem identification and design of the operational platform. Review–II: implement ingest, four stores, GraphRAG, TLP API, and analyst console against FR1–FR12. Review –III: evaluate NFR2–NFR4 and NFR8, document limitations, and show that gaps G1–G5 are closed in the running system.

10. Conclusion of Review–I

CTI is a polyglot data problem. Relationships belong on a graph; reports belong in documents; live indicators belong in TTL keys; similar prose belongs in a vector index; analyst questions belong to GraphRAG rather than ungrounded generation. The literature supports each of those sentences separately [1], [8], [9], [12], [21], [23]. What it does not yet offer is an operational workstation that holds all of them, with ingest, TLP, and auditable answers. That is the platform this project will implement. The architecture, store selection, and schemas in this report are the Review–I baseline for that build.

References

[1] W. Tounsi and H. Rais, “A survey on technical threat intelligence in the age of information sharing,” Computers & Security, vol. 72, pp. 212–233, 2018.

[2] F. Skopik, G. Settanni, and R. Fiedler, “A problem shared is a problem halved: A survey on the dimensions of collective cyber defense through security information sharing,” Computers & Security, vol. 60, pp. 154–176, 2016.

[3] OASIS Open, “STIX Version 2.1,” OASIS Standard, June 2021. [Online]. Available: https://docs.oasis-open.org/cti/stix/v2.1/

[4] B. E. Strom et al., “MITRE ATT&CK: Design and Philosophy,” The MITRE Corporation, 2020.

[5] C. Wagner, A. Dulaunoy, G. Wagener, and A. Iklody, “MISP: The design and implementation of a collaborative threat intelligence sharing platform,” in Proc. ACM Workshop on Information Sharing and Collaborative Security (WISCS), 2016, pp. 49–56.

[6] X. Liao et al., “Acing the IOC game: Toward automatic discovery and analysis of open-source cyber threat intelligence,” in Proc. ACM CCS, 2016, pp. 755–766.

[7] G. Husari, E. Al-Shaer, M. Ahmed, B. Chu, and X. Niu, “TTPDrill: Automatic and accurate extraction of threat actions from unstructured text of CTI sources,” in Proc. ACSAC, 2017, pp. 103–115.

[8] M. Stonebraker and U. Çetintemel, “‘One size fits all’: An idea whose time has come and gone,” in Proc. ICDE, 2005, pp. 2–11.

[9] P. J. Sadalage and M. Fowler, NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence. Addison-Wesley, 2012.

[10] R. Cattell, “Scalable SQL and NoSQL data stores,” ACM SIGMOD Record, vol. 39, no. 4, pp. 12–27, 2011.

[11] E. Brewer, “CAP twelve years later: How the ‘rules’ have changed,” Computer, vol. 45, no. 2, pp. 23–29, 2012.

[12] I. Robinson, J. Webber, and E. Eifrem, Graph Databases, 2nd ed. O’Reilly Media, 2015.

[13] S. Noel et al., “CyGraph: Graph-based analytics and visualization for cybersecurity,” in Handbook of Statistics, vol. 35. Elsevier, 2016, pp. 117–167.

[14] A. Piplai, S. Mittal, A. Joshi, T. Finin, J. Holt, and R. Zak, “Creating cybersecurity knowledge graphs from malware after action reports,” IEEE Access, vol. 8, pp. 211691–211703, 2020.

[15] I. Sarhan and M. Spruit, “Open-CyKG: An open cyber threat intelligence knowledge graph,” Knowledge-Based Systems, vol. 227, 2021.

[16] Filigran, “OpenCTI documentation,” 2024. [Online]. Available: https://docs.opencti.io/

[17] MongoDB, Inc., “MongoDB manual (document model),” 2024.

[18] Redis Ltd., “Redis documentation: EXPIRE, eviction, and key–value operations,” 2024.

[19] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,” IEEE Trans. Pattern Anal. Mach. Intell., vol. 42, no. 4, pp. 824–836, 2020.

[20] Qdrant, “Qdrant documentation: collections, HNSW, and payload filtering,” 2024.

[21] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in Proc. NeurIPS, 2020.

[22] V. Karpukhin et al., “Dense passage retrieval for open-domain question answering,” in Proc. EMNLP, 2020, pp. 6769–6781.

[23] D. Edge et al., “From local to global: A graph RAG approach to query-focused summarization,” arXiv:2404.16130, 2024.

[24] B. Peng et al., “Graph retrieval-augmented generation: A survey,” arXiv:2408.08921, 2024.

[25] M. Yasunaga, H. Ren, A. Bosselut, P. Liang, and J. Leskovec, “QA-GNN: Reasoning with language models and knowledge graphs for question answering,” in Proc. NAACL, 2021.

[26] Neo4j, Inc., “Neo4j Cypher manual,” 2024.

[27] ENISA, “ENISA threat landscape,” European Union Agency for Cybersecurity, 2024.

[28] Mandiant, “M-Trends,” Google Cloud, 2024.

[29] CIRCL / MISP Project, “MISP data models and objects,” 2024. [Online]. Available: https://www.misp-project.org/

[30] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” IEEE Trans. Big Data, vol. 7, no. 3, pp. 535–547, 2021.

Appendix A. Representative analyst questions

1. Which threat actors share command-and-control infrastructure, and what malware was hosted there?
2. How is APT29 connected to supply-chain compromises, and which reports describe SUNBURST?
3. Who else uses Cobalt Strike besides FIN7, and do they share any domains?
4. What destructive campaigns is Sandworm responsible for, and does it share infrastructure with APT28?
5. Find activity similar to spearphishing against government and political organisations.
6. Why is Mimikatz a weak attribution signal, and which actors still use it?

These questions are acceptance tests for the workstation: each requires Neo4j paths, MongoDB citations, Qdrant similarity, and Redis IOC state. None is answerable from a single store without loss of operational meaning.